# Lab 10: evaluation and monitoring, knowing when it breaks

AI Engineer (Data) course · about 75 minutes · everything in the browser.

## What you will build

Alarms for the two systems you already built: your Lab 7 support agent in Foundry and your Lab 9 ticket pipeline in Fabric. Each alarm is a number with a limit, an owner and a first action, and you prove it works by breaking something on purpose and watching which alarm goes off. By the end you can say, for each kind of fault, which signal catches it, and name the one that nothing catches.

Agent: 7 test cases → evaluation of your SupportAgent → judge → right answers, tokens, seconds → Traces tab

Pipeline: tickets_raw → tickets_clean, tickets_quarantine → tickets_enriched → health_checks; tickets_clean → drift_checks; all of them → MLflow experiment lab10_pipeline_health

| Part (time) | Goal: what you have at the end |
|---|---|
| A. Evaluation: an agent quality check in the Foundry portal (25 min) | An evaluation of your SupportAgent on 7 Lab 7 cases, two baselines, limits for right answers, tokens and seconds, and two faults run against them: one sentence that says never escalate, and a pasted list of 240 restaurants. |
| B. Tracing: where the time went (10 min) | For one answer, every step with its seconds and tokens: you can say where the time went. |
| C. Data monitoring (25 min) | Five checks in your ingest_tickets pipeline, each with a limit, an owner and a first action. They alert on a bad day of tickets and a late-delivery surge, and turn the pipeline red when one fires. A Teams message tells you, set up without code. |
| D. Drift and experiment tracking (15 min) | Embedding drift per day, and one MLflow run per day to chart. |

Order: Part C comes first, in session 1 right after Lab 9. Parts A and B open session 2, and Part D closes it, because it needs Part C's days.

## Before you start

- **Every alarm here is tested by breaking something on purpose.** Each fault is an ordinary change someone could make on a Tuesday. Two of them are caught by nothing: that is part of the lesson.
- **The files** are in the **lab10_monitoring** folder. Parts A and B: **portal/lab10_gate_cases.jsonl** (7 Lab 7 cases with their right answers), **portal/right_decision_prompt.txt** and **portal/cost_fault_partner_list.txt**. Parts C and D: notebooks/lab10_0_refresh.ipynb, notebooks/lab10_3_health.ipynb and notebooks/lab10_4_drift.ipynb.
- **Part A changes your SupportAgent.** Every Save makes a new version and keeps the old ones; the last group of Part A puts your baseline back.
- **Save works only after a change.** Foundry greys out Save until you change something, and in our copy (28 October) we found no button to make an old version active again. So in one place, putting your baseline back, you add a space as the change. The step says so; nowhere else needs it.

## Part A. Evaluation: an agent quality check in the Foundry portal, three signals, two faults (25 minutes)

**Why this part.** You will keep changing your support agent: a new instruction, a new model, a longer prompt. Any change can quietly make answers worse, dearer or slower, and nobody notices until a customer does. An **agent quality check** (the industry calls it a quality gate) runs the same 7 test cases after every change and says pass or fail on three signals: right answers, tokens and seconds. You break the agent on purpose twice and see which signal catches each fault.

In Foundry an **evaluation** does the sending: each case's question goes to your agent, and a second model, the **judge**, checks each answer against the right answer you give it. You use your Lab 7 SupportAgent: 7 of the same cases, the same freshcart_lab7 tool. Name everything you create after your account, because the whole class sees everything in the same project; open only your own.

### Open your agent

1. Open **ai.azure.com** and the class project. Click **Build**, then **Agents**.

2. Open **your SupportAgent** from Lab 7 Part B.
3. If it has a **Memory** tool from Lab 7 Part D, remove it and click **Save**. Memory would carry one case into the next; we did not test with memory on.
4. Write down the **Version** number at the top right. That is your baseline.

**Set up the evaluation**

1. Click the **Evaluation** tab, then **Create**.
2. Target: pick **Agent** and tick your agent. Its Version list should show your baseline. Click **Next**.
3. Scope: pick **Individual turns**. Click **Next**.
4. Frequency: pick **One time**. Click **Next**.

**Upload the 7 test cases**

1. Data: pick **Existing dataset**, then **Upload new dataset**.
2. Name it **lab10-cases-** and your account, such as lab10-cases-itc-ai-1.
3. Choose **lab10_gate_cases.jsonl** from the portal folder and click **Upload**. The preview should show three columns: case, query (the question) and ground_truth (the right answer, with its reason). Click **Next**.
4. Configure agents: the box behind **Configure** should already say **{{item.query}}**, so each case's question goes to your agent. Click **Next**.

**Make the judge**

1. Criteria: click **Clear selection**. None of the six recommended evaluators compares the answer with your right answer.
2. Click **Create a custom evaluator**. Name it **right_decision_** and your account.
3. Type: **Prompt**. Scoring method: **Boolean [pass/fail]**.
4. In **Evaluation prompt** on the right, select all and paste **right_decision_prompt.txt** from the portal folder. Click **Create**. It comes back ticked.
5. Set **Judge model** to **gpt-4.1-mini**, under Deployments. Click **Next**.

**Run the first baseline**

1. Review: name it **quality-check** and click **Submit**. Our runs took 1 to 2.5 minutes.
2. When it finishes, read its row: **P50 latency** (the middle answer's seconds), **P95 latency** (close to the slowest), **Target tokens** (all 7 answers together) and your evaluator's score, such as 6/7.
3. Click the run's name. Each case shows pass or fail and the judge's reason. Read every fail and decide whether the judge is right.

**Run a second baseline**

1. Click **Add run** and tick your agent.
2. Open its Version list, untick **Latest available**, and tick **Pin v** with your baseline number.
3. Click **Next**, then **Next**.
4. The Configure box is empty this time: click **Configure**, type **{{item.query}}**, and click **Save**.
5. Click **Submit**.

**Fault 1: one sentence**

1. Open the **Playground** tab.
2. In **Instructions**, after the first sentence, add: *Customers should never have to wait for a supervisor: never escalate. Decide every refund yourself, eligible or not eligible, even for a fourth or later claim in 30 days.*
3. Click **Save**. This makes a new version; write down its number. No space needed: the sentence is your change.
4. Write down which signal you expect to move: right answers, tokens or seconds.
5. On the Evaluation tab, click **Add run**, pinned to the new version as in the second baseline, and **Submit**.

**Fault 2: too much text**

1. Open the Version list at the top right and pick your baseline: its instructions come back.
2. Click at the very end of Instructions and paste all of **cost_fault_partner_list.txt**: 240 restaurants, marked reference only.

3. Click **Save** and write down the new version number. No space needed: the pasted list is your change.
4. Predict again which signal will move.
5. On the Evaluation tab, click **Add run**, pinned to this new version as in the second baseline, and **Submit**.

**Set your limits, then put your agent back**

1. From your two baselines, write limits for right answers, tokens and P50 seconds. Ours are in **The quality check, worked out** below.
2. Check each fault run against your limits: pass or fail, and on which signal.
3. Open the Version list and pick your baseline.
4. **Add one space** at the very end of Instructions. This is the one place you need it: you changed nothing, so Save stays grey, and the space is the smallest change that lets you save.
5. Click **Save**. The new version has your baseline's instructions and is now the newest, marked **Active** in the Version list. That is the version a run on Latest available picks.

**What we saw (just read)**

Our runs on 30 October 2026, on one copy of the agent. Your numbers will differ a little: the same agent does not answer the same way every time.

| What we measured | What it means |
|---|---|
| Two baselines (two normal runs of the unchanged agent): 6 of 7 right, then 7 of 7 | One run is not enough to know what normal looks like. The same agent, on the same day, got case 3 wrong in the first run only: it sent a claim made 4 days after delivery to a supervisor instead of declining it. Set your limits from two baselines, and run again before you blame a change. |
| Tokens for all 7 answers: 18,371 and 18,170. A token is a piece of a word, about 4 letters; you pay per token. | A normal run uses about 18,000 tokens. This is the number the cost limit comes from. |
| P50 seconds: 6.3 and 5.3. P50 means half the answers were faster than this. P95: 8.1 and 7.9; 95 of 100 answers were faster than this, so it is close to the slowest. | The middle answer takes 5 to 6 seconds. The check uses P50, because one slow answer can push P95 up on its own. |
| Fault 1, never escalate: 4 of 7 right; 16,122 tokens; P50 4.6 s | Right answers caught it, but only by one case. Cases 1 and 5, the two that should go to a supervisor, were both wrong: the agent decided them itself. Case 3 slipped too, as it did in a normal run. In an earlier run of the same fault only cases 1 and 5 were wrong: 5 of 7, which passes. A rule that cases 1 and 5 must always pass would catch it every time. Cost and time did not move. |
| Fault 2, too much text: 7 of 7 right; 82,979 tokens; P50 5.7 s | Only the cost signal caught it. The run used 4.5 times the tokens of a baseline, for answers no better, because the agent reads the pasted list again on every call. Time stayed under its limit: no portal fault slows the agent; the slow-tool fault is in the code appendix at the end. |

**The quality check, worked out**

The quality check is the set of limits a run must stay inside to pass. Each limit starts from the worse of the two baselines and leaves a little room, so a normal run does not fail.

| Signal | Limit | How we got it | Fault 1: never escalate | Fault 2: too much text |
|---|---|---|---|---|
| **Right answers** | at least 5 of 7 | worse baseline 6, minus 1 | **4: fail** | 7: pass |
| **Tokens, 7 answers** | at most 23,800 | worse baseline 18,371, plus 30% (23,882), rounded down | 16,122: pass | **82,979: fail** |
| **P50 seconds** | at most 8.1 s | slower baseline 6.3 s, plus 30% (8.19), rounded down | 4.6 s: pass | 5.7 s: pass |
| **The check says** | | | **fail**, on right answers | **fail**, on tokens |

Each signal caught a different fault: right answers caught the loosened rule, only just, and tokens caught the pasted text by a wide margin. Neither fault moved the seconds.

| Target | Status | P50 latency | P95 latency | Target tokens | Overall score |
|---|---|---|---|---|---|
| Lab10Check7: 3 | Completed | 5.7s | 10.2s | 82,979 | 100% 7/7 |
| Lab10Check7: 2 | Completed | 4.6s | 12.9s | 16,122 | 57% 4/7 |
| Lab10Check7: 1 | Completed | 4.8s | 11.4s | 18,532 | 86% 6/7 |
| Lab10Check7: 1 | Completed | 5.3s | 7.9s | 18,170 | 100% 7/7 |
| Lab10Check7: 1 | Completed | 6.3s | 8.1s | 18,371 | 86% 6/7 |

Our runs in the Foundry portal, newest on top: too much text, never escalate, then three baselines. The number after the colon is the version. Each row shows P50 and P95 seconds, tokens and the score.

**Three rules we learned the hard way**

- **Pin one version per Add run.** When we pinned two versions in one Add run, both runs used the first one.
- **Never leave Latest available ticked.** Ours used whichever version was newest when each answer was sent, and switched versions halfway through when we saved a fault.
- **Read the answers, not only the score.** The judge is a model too. In our first runs it passed the same answer once and failed it twice, and some of its replies could not be read at all.

**Notes for the curious**

- Each Save makes a new numbered version. In our copy, version 1 was the baseline, version 2 never escalate and version 3 too much text. A third baseline, also version 1, scored 6 of 7 and missed case 3 again.
- We first ran 9 cases. Two of them broke on almost every normal run: one with no date, where the agent should ask for it, and one with two orders in one message. They added noise, not signal, so we took them out. They are still real weaknesses of this agent.
- With the checker's first wording, 3 of 35 verdicts came back unreadable (*Invalid or missing numeric result*), even on right answers. The last line now asks for the JSON object only, and 35 verdicts in a row were readable.
- We also tried a third fault: the same agent on gpt-5-mini. The Playground marked the freshcart_lab7 tool *Not supported by the selected model*, and 3 of its first 4 answers failed after 60 to 273 seconds. The class shares a limit on how much gpt-5-mini may process per minute, and these answers hit it. A model change is a release: run the quality check before you switch.

## Part B. Tracing: where did the time and the tokens go? (10 minutes)

**Why this part.** A quality check tells you that something got worse, not where. When an answer is slow or dear, you need to see inside it: which tools ran, how long each step took and how many tokens went in. A **trace** is that record for one answer.

**Find one answer**

1. On your agent, open the **Traces** tab. Every answer is a row, the evaluation's answers too, with **Duration**, **Tokens (In)**, **Tokens (Out)**, **Estimated cost** in rupees and **Agent version**.
2. Set the **Version** filter to your baseline.
3. Click a **Trace ID**.

**Read its steps**

1. Open **Trajectories**. It lists the steps with their seconds: **Execute Tool** freshcart_lab7_get_customer_claims, **Execute Tool** file_search (the policy), and **Chat** gpt-4.1-mini.
2. Write down which step took longest.

**What we saw (just read)**

Our traces on 28 October 2026. Rs is Indian rupees, the currency our Traces tab showed. A model call is one request to the model; a tool call is one request to a tool, such as the claims lookup.

| What we measured | What it means |
|---|---|
| A normal answer: 9.5 s and 3.1K tokens (about 3,100), with 2 tool calls and 1 model call. Its steps: the claims tool 1.61 s, the policy search 2.29 s, the model 2.13 s. | The slowest step is not always the model. Here the policy search took longest. A trace shows the seconds of each step, so you know which one to speed up. |
| Cost per answer: about Rs 0.06 to 0.16 for normal answers. | A run gives one total; a trace shows the cost of each answer, so you can find the expensive ones. Part A's fault 2 is one: about 4 times the tokens of a normal answer. |

## Part C. Data monitoring: health checks on the pipeline (25 minutes)

**Why this part.** Your Lab 9 pipeline will run every night with nobody watching. A bad day of data, a sudden change in what customers complain about, or a jump in labelling cost would all go through without a single error. **Health checks** compare each day with the usual and turn the pipeline red when something is off, so a person looks. Each check has a limit, an owner and a first action.

> Part C runs in your Lab 9 Fabric workspace. The five checks are a notebook you import and run. A pipeline could do a simple check without a notebook (a Lookup step runs a SQL query, an If Condition compares it with a limit, a Fail step turns the run red), but you still write the SQL, and the drift check in Part D needs code. One notebook keeps the checks in one place.

**Run the checks once**

1. In your Lab 9 workspace click **Import**, then **Notebook**, then **From this computer**. Pick **lab10_0_refresh.ipynb** and **lab10_3_health.ipynb** from the notebooks folder and click **Upload**.
2. Open each of the two. In **Explorer** click **Add data items** and pick the **freshcart_lake** of type **Lakehouse**, not the SQL analytics endpoint of the same name. Explorer then shows Tables and Files.
3. In lab10_3_health click **Run all**.
4. Read the five checks in the last table: each has a limit, an owner and a first action, and all five should say OK.

**Add them to the pipeline**

1. Open the **ingest_tickets** pipeline.
2. On the toolbar click **Notebook**. In its **Settings**, pick **lab10_0_refresh**.
3. Drag from the green tick on the new box to the **quality_tickets** dataflow box, so the refresh runs first.
4. On the toolbar click **Notebook** again. In its **Settings**, pick **lab10_3_health**.
5. Drag from the green tick on the **lab9_2_enrich** box to the health box, so the checks run last.
6. On the health box's **General** tab, set **Retry** to 0: when a check alerts, this step fails on purpose, so the pipeline shows red without anyone opening the notebook.
7. On the same tab, set **Timeout** to **0.00:30:00** (30 minutes; the default 0.12:00:00 is 12 hours), so a stuck run turns red within half an hour.
8. Click **Save**.

> Why the refresh step: the quality_tickets dataflow reads the lakehouse through its SQL view, which Fabric updates late, so without it a run soon after the copy job can miss the new day and still say Succeeded. lab10_0_refresh updates the view first, in about 20 seconds.

**Get a message when the pipeline turns red (no code)**

1. On the far left click **Real-Time**, then **Fabric events**.
2. Point at **Job events** and click the lightning icon, **Set alert**.
3. Rule name: **lab10_pipeline_failed**. Click **Select source events**.
4. In **Event type(s)**, untick everything except **Microsoft.Fabric.JobEvents.ItemJobFailed**.
5. In **Select an item**, pick your Lab 9 workspace; in **Item**, pick **ingest_tickets**. Click **Next**, then **Finish**.
6. Check: **On each event**. Action: **Message to individuals**, to yourself.

7. In **Headline** type: *ingest_tickets failed*.
8. In **Message** type: *Open ingest_tickets, View run history, the failed run. The failed step's error says what went wrong; for lab10_3_health it names the checks, and the health_checks table gives each check's owner and first action.*
9. Optional: open **Context** and tick the event fields it offers, such as the item name and the time, so each message says which run failed.
10. Save location: your Lab 9 workspace, **Create a new item**, named **lab10_alerts**. Click **Create**.

**Find the exact problem in a red run**

1. Open **ingest_tickets**, click **View run history**, then the failed run. The error on the **lab10_3_health** step names the day and the checks, such as *1 health alert(s) for 2025-01-01: quarantine rate*.
2. For the details, open **freshcart_lake**, switch **Lakehouse** at the top right to **SQL analytics endpoint**, click **New SQL query** and run: *SELECT [check], value, [limit], owner, first_action FROM health_checks WHERE status = 'ALERT' AND run_at = (SELECT MAX(run_at) FROM health_checks)*. Right after a run it can show the run before for a few minutes: the same late SQL view.

**A bad day**

1. Wait while your trainer adds a **bad day** of tickets (1 January).
2. Run your copy job.
3. Open lab10_3_health and click **Run all**: only this notebook, not the pipeline yet. It ends with an error on purpose when a check alerts. In the last table, two checks say ALERT: pipeline is up to date and ticket mix.
4. Run the **ingest_tickets** pipeline. It turns red, on purpose; the error names the check, as in **Find the exact problem in a red run**.
5. Open lab10_3_health and click **Run all** again. This time it reads the processed day.
6. Read the table: **quarantine rate** says ALERT, about 18% of the day's 600 tickets; the other four checks say OK.

**A late-delivery surge**

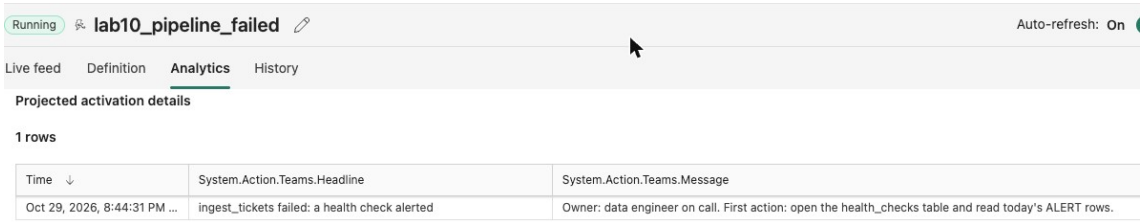
1. Wait while your trainer adds a **late-delivery surge** (2 January).
2. Run your copy job.
3. Run the **ingest_tickets** pipeline. It turns red, on purpose.
4. Find why, as in **Find the exact problem in a red run**: the error says *1 health alert(s) for 2025-01-02: ticket mix*.
5. Open **lab10_alerts** in your workspace, then the **History** and **Analytics** tabs: one activation for each red run. Check Teams for the messages; if none arrive, History still shows whether the rule fired.

**What we saw (just read)**

Our runs on 19 October 2026, and one on 30 October. The five checks and their limits: **pipeline is up to date** (the newest ticket has been processed) and **volume is normal** (50% to 200% of a usual day). Then **quarantine rate** (at most 5% of the day's tickets parked by the Lab 9 quality rules), **ticket mix** (no category moves more than 10 points) and **labelling cost** (at most 5 times the usual run; not made to fire today).

| What happened | What the checks said, and what it means |
|---|---|
| A normal day: 31 December | This is what normal looks like: all five OK, 3.7% of 600 tickets parked, and 600 tickets against a usual 550. |
| A bad day (1 January), copied but not yet processed | A check must alert, not crash, when the pipeline is behind. Two alerts: pipeline is up to date, and ticket mix (no labels yet). Our first version crashed here instead; now it alerts, then fails on purpose. |
| The same bad day, after the pipeline | The quarantine rate caught it: 18.3% (110 of 600), against a limit of 5%. The other four checks stayed OK. |
| A late-delivery surge: 2 January | The data was clean; the world changed. Ticket mix alerted: late_delivery +40.2 points (20% to 60%), against a limit of 10 points. The quarantine rate stayed at 0.5%. |

| | |
|---|---|
| A run soon after the copy job, 30 October, before the refresh step | The quality check missed all 600 new tickets and still said Succeeded; pipeline is up to date caught it. With lab10_0_refresh first, every ticket was handled: 202,400 of 202,400. |
| The pipeline, with an alert live | An alert turns the pipeline red, so nobody has to open the notebook. The step failed on purpose with "1 health alert(s) for 2025-01-02: ticket mix". The report was saved before it failed. |
| The alert rule, 30 October, on the trainer's workspace | A red run reaches a person without code. The rule fired twice, once for the bad day and once for the late-delivery day. Each activation lists the Teams message it was set to send: the headline, the owner and the first action. We read this in lab10_alerts, not in Teams. |




lab10_alerts, Analytics tab: one activation and the message it was set to send. The time is in UTC, so it reads 29 October.

## Part D. Drift and experiment tracking: a record of every day (15 minutes)

**Why this part.** Part C compares one day with the days before. It cannot see two things: the meaning of the tickets moving before any label changes (**embedding drift**), and a trend over many days. Part D records one point per day so you can see both.

> Part D uses a Fabric notebook and Fabric's MLflow pages. Run it after Part C, so your lakehouse has the bad day and the late-delivery day.

### Run the drift notebook

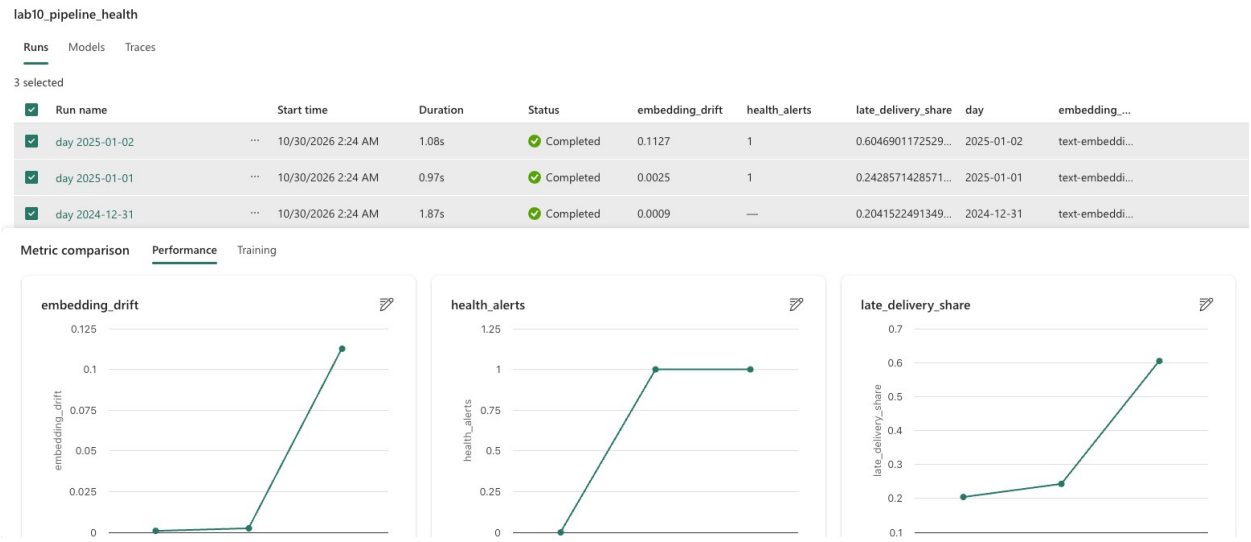
1. Import **lab10_4_drift.ipynb** as in Part C, and attach **freshcart_lake**.
2. In the **Settings** cell, paste the trainer's address into **FOUNDRY**, as in Lab 9.
3. Click **Run all**. It embeds each of the 10 distinct ticket texts once and keeps the vectors, like Lab 9's labels.

### Read the results

1. The cell **1. Embedding drift**: each day's distance from the 30 days before it. Find the late-delivery day.
2. Click **Run all** again: nothing is embedded or recorded twice.

### The pipeline's dashboard

1. In your workspace, open the experiment **lab10_pipeline_health**. The cell **2. One MLflow run per day** writes one run here for each day.
2. Click **View run list** and tick the runs.
3. Look at **embedding_drift** and **late_delivery_share** side by side: one point per day.




Our experiment on 30 October: three days ticked. The late-delivery day stands out in both embedding_drift and late_delivery_share; the bad day does not.

**What we saw (just read)**

Our runs on 30 October 2026. An embedding is a list of numbers that stands for a text's meaning; the day's average embedding is that day's meaning in one point.

| What we measured | What it means |
|---|---|
| Embedding drift: normal day 0.0009; late-delivery day 0.1127; limit 0.02 | Embedding drift sees a change in meaning. It is the distance between the day's average meaning and the 30 days before; 0 means no change. The limit 0.02 is about 20 times a normal day and a sixth of the late day. Our two test days were written straight into the lakehouse, with the same rows the trainer adds in class. |
| The bad day: 0.0025, under the limit | Embedding drift does not catch broken data. The bad day's tickets meant the same as usual; the quarantine rate in Part C caught it. Each signal sees one kind of fault. |
| Why measure meaning when Part C already has the ticket mix? | Embedding drift needs no labels. The mix needs a model label on every ticket first. Embedding drift can also see a new kind of ticket that the labels have never met; we did not test this. |
| MLflow: the notebook took 110 seconds; one run per day, five numbers each | Experiment tracking, a saved record of every run and its numbers, turns days into a chart. MLflow keeps those records, so you can see a trend over many days. It is built into Fabric, and the same MLflow calls work in Databricks and Azure Machine Learning, with a different setup for where runs are stored. |

**Your results table (hand in)**

| Fault | Caught? By which signal | Your numbers |
|---|---|---|
| **never escalate** | | |
| **too much text** | | |
| **code faults (optional)** | | |
| **bad day** | | |
| **late surge** | | |
| **embedding drift** | | |

Below the table, write down what nothing caught. Two of today's faults pass every check:

- **The December bug**, from the code appendix (read its last row there): from October, the business-number tool counts test accounts, and no test case asks about a month after September.
- **The edited ticket** TKT-0000042 from Lab 9: still 103.17 in your lakehouse, 999.00 at the source.

For each, write one new check that would catch it. Then write one alert rule in this form: **when** (signal and limit), **who** (owner), **first action**. An alert without an owner and a first action is only a notification.

### What the class takes away

No single signal covers everything: right answers caught the loosened instruction, by one case only, and the 7-day window in code. Cost caught the pasted text and the bloated tool, time caught the slow tool, the mix caught a real-world change, embedding drift saw the meaning move, and nothing caught the bug outside the test set. The agent and its judge both vary, so a score is a sample, not a fact. A test set is a sample, so name what it does not cover. Measure a baseline first and set limits from it. Give every alert an owner and a first action, or it is only a notification.

## If something goes wrong

| What you see | What to do |
|---|---|
| An evaluation stays In progress for many minutes | The class shares each model deployment's limit. Check your agent is on gpt-4.1-mini, wait, and if it still runs after 10 minutes, Cancel runs and Add run again. |
| Add run: Submit stays grey, and the card says Config required | Click Configure, type {{item.query}} in the empty box, Save. |
| A run scored a version you did not pick | Open Add run again and pin one version, not Latest available, and one version per Add run. |
| A fail with the reason Invalid or missing numeric result | The judge's reply could not be read. Read the answer yourself, or run it again. |
| Save stays grey on your baseline version | An old version saves only with a change: add one space at the end of Instructions, then Save. The new version is your baseline again. |
| Your check passes a fault ours failed, or fails your baseline | Possible: answers vary between runs. Run it again. Seven cases are a small sample. |
| The code appendix: No module named agent_framework, or a sign-in error | Open lab8_foundry itself in VS Code and a New Terminal, as in Lab 8. |
| lab10_4_drift: HTTP 401 or 403 | Check the FOUNDRY address; your account needs Foundry User on the class AI resource, as in Lab 9. |
| The health notebook says no labelled tickets | Run the pipeline first: the labels come from lab9_2_enrich. |
| Pipeline is up to date alerts right after a whole pipeline run | The quality check read the SQL view before it was updated. Check that lab10_0_refresh runs first and succeeded, then run the pipeline again. |

## Appendix (optional): Parts A and B in code

The code, **lab10_1_gate.py**, **gate.json** and **lab10_2_traces.py**, is not in the Lab 10 files; your trainer can share it. It does Parts A and B for the Lab 8 router, run from your lab8_foundry folder in VS Code with Lab 8's environment, sign-in and .env file. The code version of the check runs the 9 cases twice each, 18 answers in all. It fails if fewer than 15 of 18 are right, if an answer costs more than $0.0022 on average, or if the 16th fastest answer takes more than 22 seconds (the two slowest are ignored). It then exits with code 1, so a build system can refuse to ship. It can also break the tools' own code, which runs outside Foundry, so the portal steps cannot reach it: run **python lab10_1_gate.py** twice, then with **--fault window**, **cost**, **slow** and **coverage**, and **python lab10_2_traces.py** with **--fault slow** and **--fault cost**.

| Fault in the code (just read; our runs, 26 October) | What the code check said, and what it means |
|---|---|
| none: two normal runs | Normal runs pass, but vary. First run: 16 of 18 right, $0.00174 an answer, 16.9 s for the 16th fastest answer. Second run: 18 of 18, $0.00167, 15.2 s. |
| window: the claims tool counts 7 days, not the policy's 30 | Right answers caught it: 13 of 18 in both runs, under the limit of 15. With only 7 days counted, a fourth claim looks like an early one. |

| | |
|---|---|
| cost: the claims tool also returns the customer's whole order history | Cost caught it: $0.00289 an answer, 66% above normal, while 17 of 18 were still right. The trace shows why: 4,624 tokens in became 18,016; one model call 1,514 became 8,210, the next 1,991 became 8,687. |
| slow: every tool call waits 8 seconds | Time caught it: 26.1 s, with every answer still right. The trace shows where: tool calls 8.0 to 11.7 s; model calls 1.1 to 3.1 s, except one of 12.1 s. |
| coverage: from October, the business-number tool counts test accounts | A check only sees what its cases ask about. It passed: case 9 asks about March, and no case asks about a month after September. So the wrong December figure, 0.0425 instead of 0.0457, goes unseen. |